

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT- 03

NAME – HARI RISHIKA KOMATI

ENROLLMENT NUMBER – 2403A51083

COURSE NAME – AI&DL

DATE – 12-08-26

Lab/Exp 3: Implementing an ANN for Binary Classification -e.g., Logistic Regression equivalent

Kaggle Dataset Link: <https://www.kaggle.com/datasets/yasserh/breast-cancer-dataset>

Tasks:

1. Load and preprocess the Breast Cancer dataset
 2. Build the required PyTorch model/program.
 3. Train/execute the model.
 4. Evaluate using suitable metrics.
 5. Visualize results using plots where applicable.
 6. Write a brief conclusion comparing observations.
-

CODE –

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)

from google.colab import files

print("Please upload your Breast Cancer CSV dataset:")
uploaded = files.upload()

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print("\nDataset loaded successfully!")

print("\nFirst 5 rows:")
display(df.head())

print("\nDataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())
```

```
▶ # Remove unnecessary columns if they exist

columns_to_drop = []

if 'id' in df.columns:
    columns_to_drop.append('id')

if 'Unnamed: 32' in df.columns:
    columns_to_drop.append('Unnamed: 32')

df = df.drop(columns=columns_to_drop)

print("\nAfter removing unnecessary columns:")
print(df.shape)

# Convert diagnosis into binary values
# M = 1 (Malignant)
# B = 0 (Benign)

if 'diagnosis' in df.columns:
    df['diagnosis'] = df['diagnosis'].map({
        'M': 1,
        'B': 0
    })

elif 'Diagnosis' in df.columns:
    df['Diagnosis'] = df['Diagnosis'].map({
        'M': 1,
        'B': 0
    })
    df.rename(columns={'Diagnosis': 'diagnosis'}, inplace=True)

else:
    print("Diagnosis column not found.")
    print("Available columns:", df.columns.tolist())

# Check missing values

print("\nMissing values:")
print(df.isnull().sum())
```

```
# Remove rows containing missing values

df = df.dropna()

print("\nDataset after preprocessing:")
print(df.shape)

X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

print("\nNumber of features:", X.shape[1])
print("Number of samples:", X.shape[0])

print("\nClass distribution:")
print(y.value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train_tensor = torch.tensor(
    X_train,
    dtype=torch.float32
)

X_test_tensor = torch.tensor(
    X_test,
    dtype=torch.float32
)

y_train_tensor = torch.tensor(
    y_train.values,
    dtype=torch.float32
).reshape(-1, 1)
```

```

print("\nTensor shapes:")
print("X_train:", X_train_tensor.shape)
print("y_train:", y_train_tensor.shape)

class BreastCancerANN(nn.Module):

    def __init__(self, input_size):
        super(BreastCancerANN, self).__init__()

        self.network = nn.Sequential(

            # Input layer -> Hidden layer
            nn.Linear(input_size, 32),
            nn.ReLU(),

            # Hidden layer
            nn.Linear(32, 16),
            nn.ReLU(),

            # Output layer
            nn.Linear(16, 1)
        )

    def forward(self, x):
        return self.network(x)

# Create model

input_size = X_train.shape[1]

model = BreastCancerANN(input_size)

print("\nANN Model:")
print(model)

criterion = nn.BCEWithLogitsLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

epochs = 100

train_losses = []
train_accuracies = []

```

```
for epoch in range(epochs):

    # Training mode
    model.train()

    # Forward pass
    outputs = model(X_train_tensor)

    # Calculate loss
    loss = criterion(
        outputs,
        y_train_tensor
    )

    # Backpropagation
    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    # Convert logits into probabilities
    probabilities = torch.sigmoid(outputs)

    # Convert probabilities into 0 or 1
    predictions = (
        probabilities >= 0.5
    ).float()

    # Calculate accuracy
    accuracy = (
        (predictions == y_train_tensor)
        .float()
        .mean()
        .item()
    )

    train_losses.append(loss.item())
    train_accuracies.append(accuracy)

    # Print progress
    if (epoch + 1) % 10 == 0:

        print(
            f"Epoch [{epoch+1}/{epochs}] "
            f"Loss: {loss.item():.4f} "
            f"Accuracy: {accuracy*100:.2f}%"
        )
```

```
model.eval()

with torch.no_grad():

    # Get raw predictions
    test_outputs = model(X_test_tensor)

    # Convert logits to probabilities
    test_probabilities = torch.sigmoid(
        test_outputs
    )

    # Convert probabilities to class predictions
    test_predictions = (
        test_probabilities >= 0.5
    ).int().numpy().flatten()

# Actual values
y_test_actual = y_test.values

accuracy = accuracy_score(
    y_test_actual,
    test_predictions
)

precision = precision_score(
    y_test_actual,
    test_predictions
)

recall = recall_score(
    y_test_actual,
    test_predictions
)

f1 = f1_score(
    y_test_actual,
    test_predictions
)
```

```

print("\n=====")
print("MODEL PERFORMANCE")
print("=====")

print(f"Accuracy : {accuracy * 100:.2f}%")
print(f"Precision : {precision * 100:.2f}%")
print(f"Recall : {recall * 100:.2f}%")
print(f"F1 Score : {f1 * 100:.2f}%")

# =====
# 12. CLASSIFICATION REPORT
# =====

print("\nClassification Report:")

print(
    classification_report(
        y_test_actual,
        test_predictions,
        target_names=[
            "Benign",
            "Malignant"
        ]
    )
)

cm = confusion_matrix(
    y_test_actual,
    test_predictions
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "Benign",
        "Malignant"
    ],
    yticklabels=[
        "Benign",
        "Malignant"
    ]
)

```

```
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - ANN")

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    train_losses,
    label="Training Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss vs Epochs")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    train_accuracies,
    label="Training Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training Accuracy vs Epochs")

plt.legend()
plt.grid(True)

plt.show()

test_probabilities_np = (
    test_probabilities
    .numpy()
    .flatten()
)

fpr, tpr, thresholds = roc_curve(
    y_test_actual,
    test_probabilities_np
)
```

```

roc_auc = auc(
    fpr,
    tpr
)

plt.figure(figsize=(7, 5))

plt.plot(
    fpr,
    tpr,
    label=f"ANN (AUC = {roc_auc:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.legend()
plt.grid(True)

plt.show()

print(f"\nROC-AUC Score: {roc_auc:.4f}")

print("\n=====")
print("SAMPLE PREDICTIONS")
print("=====")

for i in range(
    min(10, len(test_predictions))
):
    actual = (
        "Malignant"
        if y_test_actual[i] == 1
        else "Benign"
    )

    predicted = (
        "Malignant"
        if test_predictions[i] == 1

```

```

    probability = test_probabilities_np[i]

    print(
        f"Sample {i+1}: "
        f"Actual = {actual}, "
        f"Predicted = {predicted}, "
        f"Probability = {probability:.4f}"
    )

print("\n=====")
print("CONCLUSION")
print("\n=====")

```

OUTPUT –

Dataset loaded successfully!

First 5 rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	radius_worst	texture_worst	perimeter_worst	area_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38	17.33	184.60	2019.0
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99	23.41	158.80	1956.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57	25.53	152.50	1709.0
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91	26.50	98.87	567.7
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54	16.67	152.20	1575.0

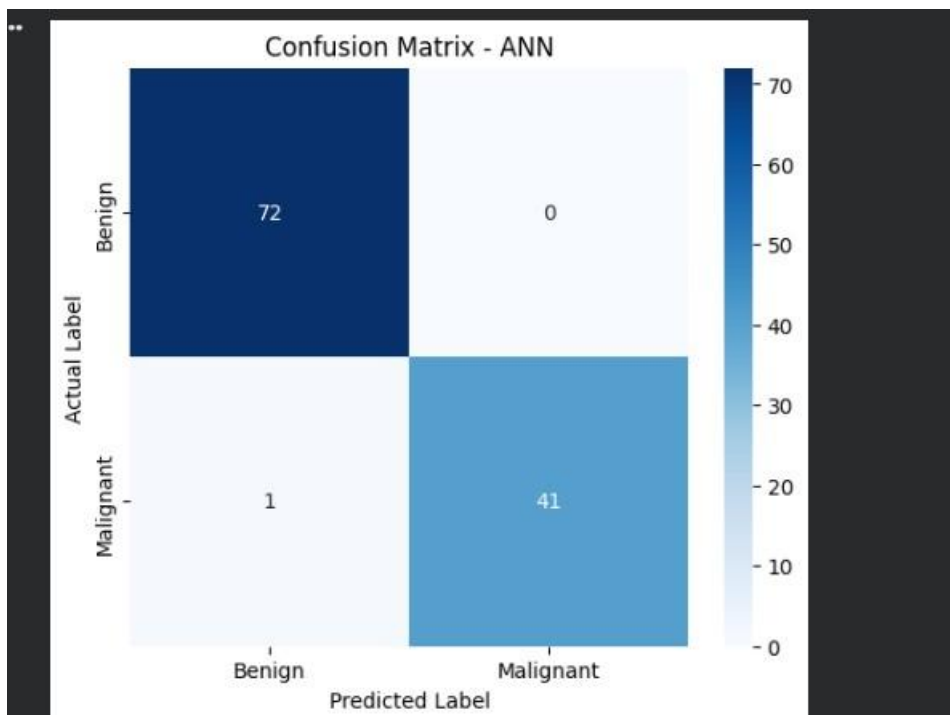
5 rows x 32 columns

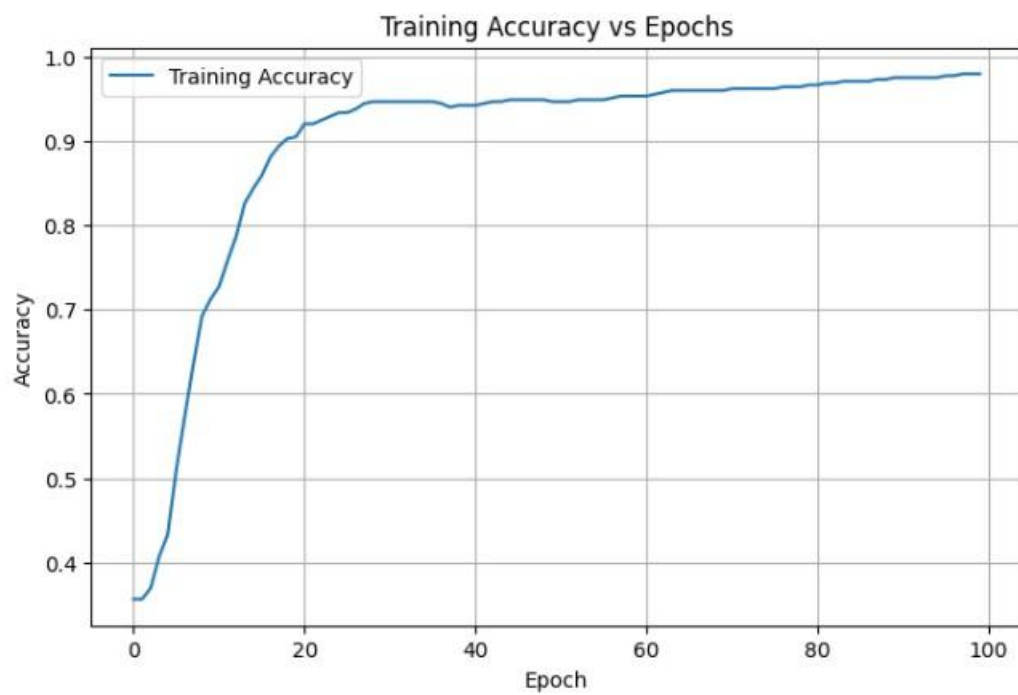
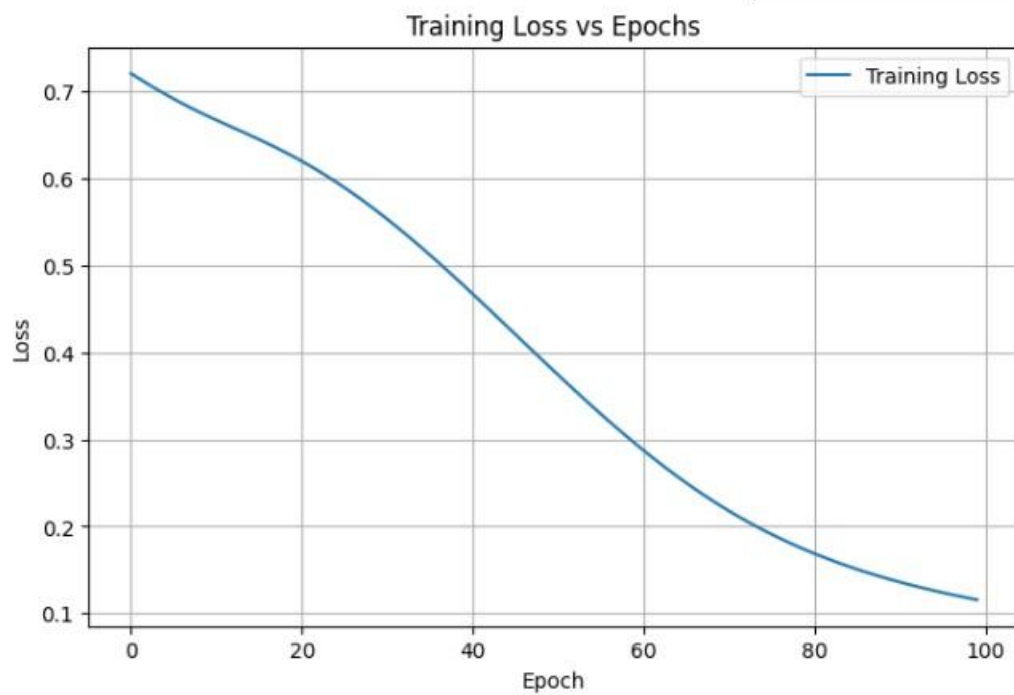
```
*** =====
MODEL PERFORMANCE
=====
Accuracy : 99.12%
Precision : 100.00%
Recall : 97.62%
F1 Score : 98.80%

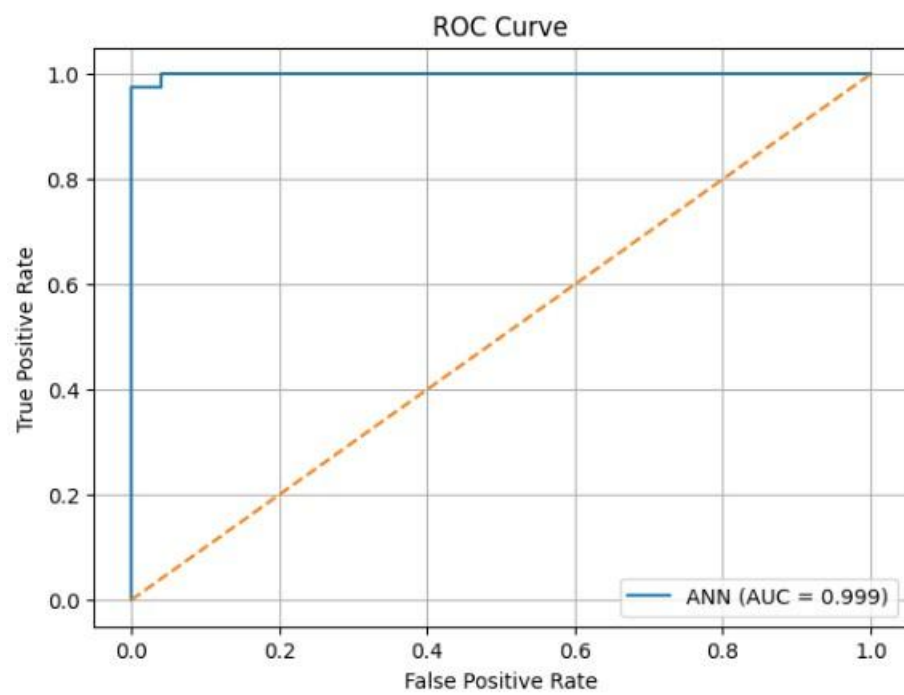
Classification Report:
              precision    recall  f1-score   support

   Benign         0.99        1.00        0.99         72
  Malignant        1.00        0.98        0.99         42

   accuracy          0.99          0.99          0.99        114
  macro avg          0.99          0.99          0.99        114
 weighted avg          0.99          0.99          0.99        114
```







ROC-AUC Score: 0.9990

```
=====
*** SAMPLE PREDICTIONS
=====
Sample 1: Actual = Benign, Predicted = Benign, Probability = 0.0190
Sample 2: Actual = Malignant, Predicted = Malignant, Probability = 0.9992
Sample 3: Actual = Benign, Predicted = Benign, Probability = 0.2824
Sample 4: Actual = Malignant, Predicted = Malignant, Probability = 0.6363
Sample 5: Actual = Benign, Predicted = Benign, Probability = 0.2609
Sample 6: Actual = Benign, Predicted = Benign, Probability = 0.1106
Sample 7: Actual = Malignant, Predicted = Malignant, Probability = 0.7949
Sample 8: Actual = Benign, Predicted = Benign, Probability = 0.0286
Sample 9: Actual = Benign, Predicted = Benign, Probability = 0.0203
Sample 10: Actual = Benign, Predicted = Benign, Probability = 0.0833
```

=====

CONCLUSION

=====

The Artificial Neural Network (ANN) was successfully implemented using PyTorch for binary classification of breast cancer tumors.

The dataset was preprocessed by removing unnecessary columns, converting the diagnosis labels into binary values, splitting the dataset into training and testing sets, and standardizing the features.

The ANN consists of an input layer, two hidden layers with ReLU activation, and one output neuron.

BCEWithLogitsLoss was used as the loss function and Adam was used as the optimizer.

The model was evaluated using Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and ROC-AUC.

Overall, the experiment demonstrates how an ANN can be implemented in PyTorch for a binary classification problem.